

ISP Final Exam - Paper 2

Course: INTRODUCTION TO SYSTEMS PROGRAMMING - A

Date: Thursday, July 9, 2026

Total Points: 100

Exercise 1 (5 points)

Consider the following recursive function:

```
static int gcd(int a, int b) {  
    if (b == 0) {  
        return a;  
    } else {  
        return gcd(b, a % b);  
    }  
}
```

What does `gcd(48, 18)` return?

- a) It will return 2
 - b) It will return 6
 - c) It will return 12
 - d) It will return 18
 - e) It will produce a runtime error due to stack overflow
 - f) It will never return because the recursion doesn't terminate
-

Exercise 2 (6 points)

Consider the following recursive function:

```
static int f(int n) {  
    if (n <= 1) {  
        return n;  
    } else {  
        return f(n - 1) + f(n - 2);  
    }  
}
```

What does `f(6)` return?

- a) It will return 5
- b) It will return 8
- c) It will return 13

- d) It will return 21
- e) It will produce a runtime error because of stack overflow
- f) It will never return because the recursion doesn't terminate

Exercise 3 (8 points)

Complete the following recursive function that calculates the power of a number: x^n . You may assume $n \geq 0$.

```
static int power(int x, int n) {  
    // TODO: Complete this function  
}
```

Exercise 4 (8 points)

Complete the following recursive function that generates all substrings of a given string. For example, `allSubstrings("abc")` should return `["", "a", "b", "c", "ab", "ac", "bc", "abc"]`.

```
ArrayList<String> allSubstrings(String s) {  
    // TODO: Complete this function  
}
```

Exercise 5 (6 points)

Consider the following class representing a node in a singly-linked list:

```
class Node {  
    int element;  
    Node next;  
}
```

Complete the following function `findLast` that returns the last element in the linked list. The function receives a node which is part of a linked list (not necessarily the first node). You must first find the beginning of the list.

```
static int findLast(Node p) {  
    // TODO: Complete this function  
}
```

Exercise 6 (8 points)

Complete the `addAtIndex` method of the linked list class. The method inserts a new node with element `x` at the given position `i`. Positions range from 0 to `n` (inclusive), where `n` is the length of the list. You may assume $0 \leq i \leq n$.

```
class List {
    private Node first;

    void addAtIndex(int i, int x) {
        // TODO: Complete this method
    }

    private class Node {
        int element;
        Node next;
    }
}
```

Exercise 7 (6 points)

Consider the following binary tree representation:

```
class TreeNode<T> {
    T key;
    TreeNode<T> leftChild;
    TreeNode<T> rightChild;
}
```

Complete the following function `treeHeight` that returns the height of the binary tree (the number of nodes on the longest path from root to leaf). An empty tree has height 0.

```
static <T> int treeHeight(TreeNode<T> root) {
    // TODO: Complete this function
}
```

Exercise 8 (8 points)

Complete the following function `collectLeaves` that returns an `ArrayList` containing all the leaf nodes (nodes with no children) in the binary tree.

```
static <T> ArrayList<T> collectLeaves(TreeNode<T> root) {
    // TODO: Complete this function
}
```

```
}
```

Exercise 9 (6 points)

Consider the following classes:

```
class X {  
    void method() {  
        System.out.print("X");  
    }  
}  
  
class Y extends X {  
    void method() {  
        System.out.print("Y");  
        super.method();  
    }  
}  
  
class Z extends Y {  
    void method() {  
        System.out.print("Z");  
    }  
}
```

What will this code print when executed from the main method?

```
X obj1 = new Z();  
Y obj2 = new Y();  
obj1.method();  
obj2.method();
```

- a) It will print ZY
- b) It will print ZX
- c) It will print YXYX
- d) It will print ZYX
- e) It will print ZXY
- f) It will not run because `obj1` is of type `X` which does not have a `method()`

Exercise 10 (6 points)

Consider the following classes:

```
class Vehicle {
    void start() {
        System.out.print("Vehicle");
    }
}

class Car extends Vehicle {
    void start() {
        System.out.print("Car");
    }
}

class Bike extends Vehicle {
    void start() {
        System.out.print("Bike");
    }
}
```

What will this code print?

```
Vehicle v1 = new Car();
Vehicle v2 = new Bike();
v1.start();
v2.start();
```

- a) It will print VehicleVehicle
- b) It will print CarBike
- c) It will print VehicleBike
- d) It will print CarVehicle
- e) It will produce a runtime error
- f) It will not compile

Exercise 11 (8 points)

Define a class `LimitedCounter` that extends `Counter`. The `increment()` method should only increment the counter up to a maximum value (specified in the constructor). If the counter is already at the maximum, `increment()` should have no effect.

```
class Counter {
    protected int value;

    Counter() {
        this.value = 0;
    }

    void increment() {
```

```
        this.value++;
    }

    int getValue() {
        return this.value;
    }
}

// TODO: Define LimitedCounter class
```

Exercise 12 (6 points)

Consider the following interface:

```
interface Predicate<T> {
    boolean test(T element);
}
```

Define a class `EvenPredicate` that implements `Predicate<Integer>` and tests whether an integer is even.

```
// TODO: Define EvenPredicate class
```

Exercise 13 (8 points)

Complete the following function `allMatch` that receives a `Predicate` and an `ArrayList`, and returns `true` if all elements in the list satisfy the predicate, and `false` otherwise.

```
static <T> boolean allMatch(Predicate<T> predicate, ArrayList<T> list) {
    // TODO: Complete this function
}
```

Exercise 14 (6 points)

Define a class `FibonacciIterator` that implements `Iterator<Integer>` and generates the Fibonacci sequence: 0, 1, 1, 2, 3, 5, 8, 13, ...

```
// TODO: Define FibonacciIterator class
```

Exercise 15 (5 points)

Which of the following is the correct way to handle resources properly in Java?

- a) Using `try` block without `catch` or `finally`
 - b) Using `try-with-resources` statement
 - c) Using `Thread.sleep()` to wait for resources
 - d) Using `System.gc()` to free resources
 - e) Using `finalize()` method in the class
 - f) Using `Runtime.getRuntime().freeMemory()`
-

Exercise 16 (6 points)

Consider the following code that writes to a file:

```
FileOutputStream fos = new FileOutputStream("output.txt");
fos.write(65);
fos.write(66);
fos.write(67);
fos.close();
```

What is the problem with this code?

- a) The file name must be "output.dat"
- b) The `write()` method should receive a `String` parameter
- c) It should use `FileWriter` instead of `FileOutputStream`
- d) The code should handle `IOException` exceptions
- e) It should use `flush()` before `close()`
- f) There is no problem; the code is correct